
RECURSIVE ORGANIZATION IMPROVEMENT: A MODELING SPECIFICATION FOR HUMAN–AGENT ORGANIZATIONS

MODELING SPECIFICATION AND MECHANISM STUDY

Zilong Wang
CataX AI
wangzilong@cata-x.ai

September 30, 2026

ABSTRACT

Stronger AI agents do not automatically produce better organizations: teams must also learn which work arrangements to retain and when to reconsider them. We propose a modeling specification for recursive organization improvement and evaluate it through an executable checker, a public-record mapping, and controlled simulation. The specification connects actor-visible histories, organizational memory, decision rights, and evidence-carrying change contracts. The mechanism study crosses six decision rules, three memory conditions, and three task environments under fixed resource ceilings. In a stationary environment, cumulative evidence raises balanced evaluation’s normalized net value per task from 0.45224 to 0.48007. Repeated reassessment’s disadvantage relative to this comparator falls from 0.01702 with reset evidence to 0.00007 with cumulative evidence. A reversal of the best workflow reveals the opposite cost: indefinite retention delays adaptation, while a finite window restores eventual performance at a transition cost. In exploratory controls, matching trial acquisition and label reuse reduces the apparent reassessment gain from 0.00607 to 0.00191. Program replacement adds no stable benefit across the tested reversal times. The study identifies evidence acquisition, reuse, and timely updating as mechanisms that must be separated from evaluator replacement when assessing organizational improvement.

Keywords human–AI interaction · multi-agent organizations · organizational learning · workflow adaptation · recursive improvement · evaluation

1 Introduction

Consider a software team that uses AI agents to draft pull requests (PRs). Faster drafting can lengthen the review queue. The team may respond by routing routine changes to automated checks and reserving human review for difficult cases. To evaluate this change, it must decide which PRs to audit, how to combine new observations with earlier evidence, and who may revise the routing or audit rule. An apparent gain can reflect a better workflow, a different sample of reviewed cases, or more effective reuse of evidence. How can the team distinguish these explanations and retain changes that improve delivery?

We propose a modeling specification for this problem and evaluate it with an executable checker, a mapping of public PR records, and a controlled mechanism study. The running PR example connects the specification’s fields to concrete organizational decisions; the simulation isolates evaluation and memory using prescribed task distributions.

Task-level productivity studies and studies of human–AI collaboration address different parts of this problem. Experiments on professional writing and evidence from customer support document productivity effects in particular settings [11, 39]; a meta-analysis finds substantial variation in the performance of human–AI combinations relative to their constituent members [47]. Organization design supplies the intervening objects: the division and allocation of work, dependencies, and integration of effort [35, 42]. Complementary organizational investments can also affect the returns

to a new technology [10, 37]. Consequently, comparing agent models while holding the organization fixed answers a different question from comparing organizations that can revise their workflows.

Workflow revision creates a measurement problem of its own. A new review policy changes both which cases receive attention and which failures become visible. If the same selected cases determine whether that policy is retained, organizational learning can favor a locally successful but globally harmful arrangement. An improvement procedure therefore needs an explicit account of the evidence it can acquire, the changes it can authorize, and the costs it incurs. The procedure may itself become a target of revision.

We call this problem *recursive organization improvement* (ROI; distinct from return on investment): evaluating and retaining changes to organizational arrangements, including changes to the procedures through which those arrangements are improved. The research object is the human-agent organization. Agent capability, human judgment, coordination, evidence access, and authority can change at different rates and under different constraints.

This paper contributes a modeling specification that connects organizational state, actor-visible event histories, and evidence-carrying change contracts. Its distinctive organizing choice is to make the object of a revision and the procedure used to evaluate it jointly explicit. The specification supports reconstructing a decision from its available evidence, checking whether a patch meets declared admission conditions, and comparing retained changes under a stable outcome criterion. These are concrete modeling obligations against which an implementation can be inspected.

We study one mechanism in depth: how an organization discovers, retains, and updates evaluation evidence. The experiment crosses evidence-reset, cumulative, and finite-window memory with fixed evaluation programs, one-time discovery followed by budget reallocation, and repeated program assessment. A template-ranking reversal makes old evidence substantively obsolete. Matching resource ceilings and the acquisition process allows us to distinguish the effect of remembering evidence from the effect of replacing an evaluator. The results show that memory can remove an apparent cost of repeated improvement in a stable environment, yet delay adaptation after change. Additional acquisition-matched controls substantially narrow the gain attributable to program replacement. The specification’s role is to make these organizational interventions and their evidence requirements explicit.

The argument follows three stages. Discovery produces information and candidate arrangements. Retention turns observations into reusable organizational knowledge. Reassessment tests whether that knowledge and the arrangements built on it remain useful. These stages connect a stronger agent’s task performance to a team’s ability to improve its work over time.

2 Conceptual foundations and modeling requirements

2.1 Organizational learning and explicit organizations

Organizational learning connects experience to retained routines [32, 36]. The distinction between the abstract understanding of a routine and its situated performance helps explain how routines can generate both stability and change [19]. Deliberate learning adds articulation and codification of experience [50], while double-loop learning addresses governing assumptions and values [4]. Engelbart’s improvement-of-improvement distinction supplies a close conceptual antecedent for examining the procedures that generate organizational change [18]. ROI translates this question into versions, events, change targets, and outcome comparisons.

Human-agent organizations also require an account of interdependence. Mixed-initiative interaction and levels of automation distinguish who initiates or performs a function [26, 40]; Coactive Design places the support for interdependent activity at the center of system design [29]. Appropriate reliance concerns when a person should use automated advice [31]. These choices alter information exposure, decision rights, and resource use, so they belong in the organizational model rather than being inferred from an actor’s capability score.

Existing formalisms already supply much of the representational machinery. MOISE+ models structural, functional, and deontic organization; OperA represents organizational interaction and agent autonomy [17, 28]. Process mining uses event data for process discovery, conformance checking, and enhancement [48]. The proposed specification connects these objects to the evaluation and retention of changes through the relations in [table 1](#). A sufficiently annotated event log or state-transition model can encode the same relations. The contribution is their explicit organization into a comparison contract, whose completeness and execution can be checked.

Automated agent-system design and workflow optimization provide candidate-generation mechanisms [27, 49]; linguistic feedback can support behavioral revision [44]. In ROI, these mechanisms are embedded in an organization with human roles, restricted evidence, and retention decisions. The central comparison concerns organizational outcomes under specified changes, whether the candidate was written by a person, generated by an agent, or selected from a finite catalog.

Table 1: Connections to established approaches. The right column identifies the records required for an ROI comparison, rather than an expressiveness ranking.

Approach	Established modeling object	Required connection for a change comparison
MOISE+; OperA	Roles, missions, norms, interactions	Target-specific authority, prior version, evidence and comparator for a revision.
Process mining	Cases, activities, event order, conformance	Organization/procedure versions, actual observation access, and sampling coverage.
Organizational learning	Experience, routines, retention	Link a retained routine to the experiment and criterion used to accept it.
Agent/workflow search	Executable candidates and evaluations	Human resources, rights, generated-output costs, and revision of the evaluator.
Generic event log	Arbitrary recorded attributes	All the same relations, if explicitly supplied; missing fields remain unknown.

2.2 From development practices to measurable arrangements

Waterfall-like staging, agile development, and Extreme Programming suggest different settings for dependencies, batch size, feedback delay, test placement, and ownership. Royce’s original account includes iteration and the risks of late system testing [43]; agile principles emphasize frequent delivery and reflection [8]; XP provides more concrete testing, pairing, and integration practices [2]. These labels identify families of configurations rather than an ordering of organizational quality. A team can retain release gates while shortening internal feedback cycles, or introduce agents into an unchanged approval pipeline.

For example, let drafting, review, and integration capacities be 4, 5, and 9 accepted-task equivalents per day. A deterministic serial capacity bound is their minimum. Doubling drafting alone raises the bound from 4 to 5; raising review capacity to 8 at the same time raises it to 8. The factorial interaction is $8 - 5 - 4 + 4 = 3$. This standard bottleneck calculation [3, 33] isolates why technical and workflow changes can be complementary. Rework, variable quality, and demand must be modeled separately in an empirical application. Our specification records the fields needed to distinguish such joint changes from an increase in task-level speed.

3 A modeling specification for recursive organization improvement

3.1 State, improvement procedure, and change boundary

An instance declares a task population, environment process, outcome criterion, and assessment horizon. The environment generates tasks and outcomes; actor policies receive only the observations assigned to them. At event index t , define

$$\mathcal{O}_t = (X_t, D_t, L_t, \Phi_t, \Pi_t, M_t), \quad U_t = (g_t, a_t, e_t, s_t, r_t). \quad (1)$$

The organization state $\mathcal{O}$ describes who does the work and what they can know or change. The procedure U describes how the team proposes, tests, selects, and retains changes. Table 2 maps their components to the PR example. Actors also have task-specific capability profiles; approval rights are recorded separately.

Definition 1 (Declared boundary). *A boundary $\mathcal{B}$ partitions fields into operational arrangements, improvement-procedure fields, and externally fixed fields for a stated comparison. It identifies authorized editors, the evaluation horizon, and the outcome criterion.*

An execution event changes work artifacts. An operational adaptation changes an arrangement under the declared improvement procedure. A procedure revision changes a field of U and evaluates its consequences for subsequent organizational changes or outcomes. For instance, assigning a PR to a reviewer is execution; replacing its routing rule is operational adaptation; replacing the sampling program used to evaluate routing changes is procedure revision. A fixed outer selector may govern all three. The recursive designation identifies a change target relative to $\mathcal{B}$, not a representation-independent property of software.

This boundary also resolves overlapping implementations. Φ applies the current visibility rules, while a requests evidence for a change evaluation. If one patch affects both, the implementation records the coupled fields and charges its costs once. If the outcome criterion changes, its version changes too; comparisons under the old and new criteria are reported separately.

Table 2: Core notation, illustrated by PR review routing.

Symbol	Meaning	PR example
X	Actors, roles, tasks, versioned artifacts	Reviewer, agent, PR, test report.
D	Task and resource dependencies	Approval depends on tests and reviewer availability.
L	Decision rights	Maintainer may change routing; review board may change auditing.
Φ	Observation availability	Which test results or private assessments each actor can see.
Π	Interaction and allocation protocols	Assignment order, review sequence, and workload allocation.
M	Retained evidence and versions	Audit labels, trial scores, and the adopted routing rule.
$U = (g, a, e, s, r)$	Generate, acquire, evaluate, select, retain	Propose a rule; audit PRs; score and select it; monitor continued use.
$\mathcal{B}$	Declared change boundary	Routing and auditing may change; outcome criterion stays fixed.
$\mathcal{H}_{i,t}$	Evidence visible to actor i before event t	Reports available before approval.
$p, \mathcal{T}, H$	Change contract, event trace, evaluation horizon	Recorded routing change and its follow-up period.

3.2 Events and actor-visible histories

For the PR team, an event may be a reviewer assignment, a private assessment, or a revealed test report. An event z records an identifier, logical order, actor, action, input/output artifact versions, recipients, resource use, and organization version. Actions include assignment, commitment, reveal, critique, test, escalation, approval, and patch application. Logical order represents dependency; physical timestamps can additionally represent concurrency and elapsed time.

Let $\mathcal{H}_{i,t}$ contain actor i 's permitted initial information and artifacts revealed to that actor before event t . A policy satisfies

$$b_{i,t} \sim \pi_i(\cdot \mid \mathcal{H}_{i,t}, \mathcal{O}_{i,t}^{\text{vis}}), \quad (2)$$

where $\mathcal{O}_{i,t}^{\text{vis}}$ is the actor-visible projection of the organization state. Thus a simulator can score a decision using a hidden true error rate while the decision maker receives sampled labels only. A seed supports reproducibility without granting actors access to each other's random choices.

The ordering of commitment and reveal events represents human-first, agent-first, and parallel assessment. Parallel commitments preserve the opportunity for private judgments, while shared training data, common tools, or correlated task difficulty can still induce dependent errors. Statistical dependence is a joint property requiring measurements or explicit assumptions; it cannot be inferred from separate model names. Likewise, a public review record establishes that a review was published, not that another actor read it before deciding.

3.3 Change contracts and retention

A proposed routing change carries a contract: a record linking the change to its evidence, comparator, cost, and follow-up rule. Formally,

$$p = (\tau, v, \delta, \mathcal{E}, \mathcal{C}, c_p, H, \mathcal{R}_p). \quad (3)$$

τ identifies the target and proposer; v is the required input version; δ is the transformation; $\mathcal{E}$ identifies available evidence and its provenance; $\mathcal{C}$ states the comparator, population, and criterion; c_p specifies resource accounting; H is the horizon; and $\mathcal{R}_p$ specifies retention, monitoring, and rollback conditions. Contract fields may refer to separately stored artifacts. Their relationships, summarized in table 3, determine whether a comparison can be reconstructed.

Admission checks authorization, versions, available evidence, prerequisites, and resources. Efficacy is a subsequent outcome comparison. These predicates have different witnesses: permission records can justify admitting a trial, while a trial's results can justify retaining its policy. Rejected and unsuccessful revisions remain in memory because they consume resources and influence future search.

For external criterion J and specified comparator 0, the effect of a patch is

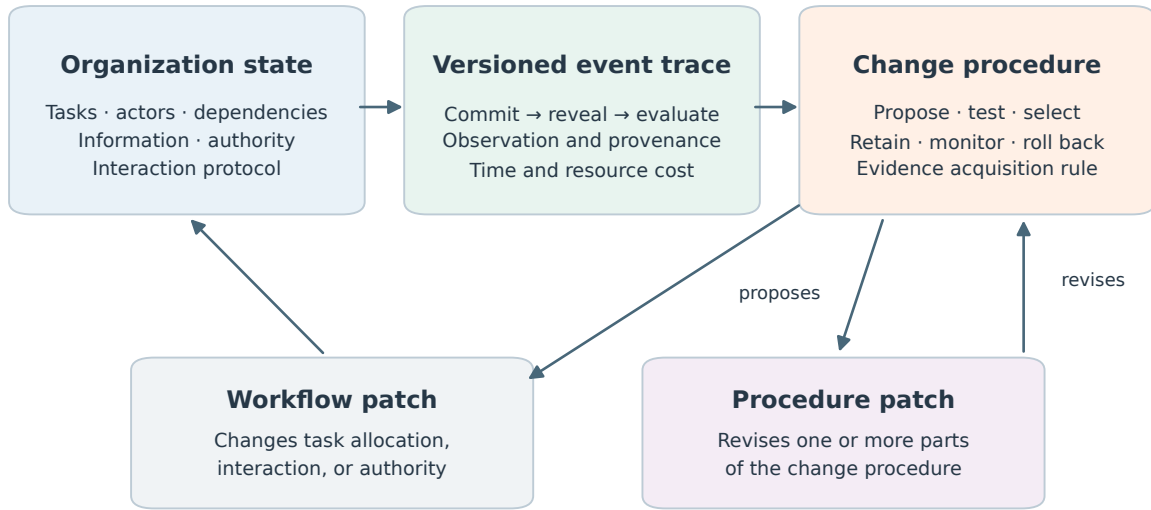
$$\Delta_H(p) = \mathbb{E}[J(\mathcal{T}_{t:t+H}^p) - J(\mathcal{T}_{t:t+H}^0)] - C_H(p). \quad (4)$$

C_H contains costs not already included in J . Randomized comparisons, controlled simulation, or explicit causal assumptions are needed to estimate the two potential outcomes. Outcomes without defensible common units remain a

Table 3: Specification obligations and observable failure conditions. Unknown information is distinguished from a violated condition.

Relation	Required condition	Failure or missing record
Actor–evidence	Each decision input was available to its actor beforehand.	Hidden/future input; unknown exposure history.
Patch–state	The target version exists and matches the expected version.	Stale patch; unversioned configuration.
Actor–target	The actor holds the applicable editing right at that version.	Unauthorized change; unknown historical permission.
Evidence–population	Sampling scope and label source support the stated population.	Uncovered stratum; unknown inclusion mechanism.
Outcome–criterion	Both arms use the declared criterion and horizon.	Changed success definition; censored follow-up.
Cost–resource	Search, outputs, labels, and transitions enter one ledger.	Omitted counterfactual output; duplicate charge.
Decision–memory	Accepted and rejected proposals retain evidence and versions.	Retained policy without its validation domain.

A declared boundary makes the level of change inspectable



External criterion, authority limits, and evaluation horizon are declared separately.

Figure 1: The modeling specification connects organization state, event histories, and improvement procedures. A revision names its target, evidence, authority, and external evaluation criterion.

vector with declared constraints. In the experiment below, the net-value criterion incorporates every modeled monetary cost, so no additional cost is subtracted in [equation \(4\)](#).

[Figure 1](#) connects execution to this revision loop. A software team might first change review routing, then discover that its evaluations cover only human-reviewed PRs. A second patch can alter evidence acquisition to include routine changes. That second patch is evaluated through later routing decisions and their outcomes; the mere act of adding an audit does not establish its net benefit.

3.4 Organizational memory as a testable design choice

The memory field M separates three retained objects: workflow evidence, evaluation-program evidence, and adopted arrangements. Keeping a program name without its supporting observations preserves a decision but does not accumulate statistical knowledge. Keeping observations indefinitely can preserve knowledge about a population that no longer exists. An instance therefore records evidence provenance, applicability, and expiry alongside the retained policy.

The retention component $U.r$ specifies when evidence or arrangements are reconsidered. A fixed window is one such rule; changing the rule is a procedure revision at the declared boundary. In the mechanism study, memory rules are experimentally assigned and held fixed within a run. Evaluation programs may then change inside each memory condition. This factorial construction separates a design intervention on retention from the execution of a program-selection procedure.

Evidence generated during organizational search can serve more than one decision. A rejected program trial may still produce valid labels about a workflow. The trace records the labels once and reveals them to the permitted downstream evaluator; their cost is charged once. Comparing a policy that reuses those labels with one that discards them tests an information-transfer rule. Comparing two policies with the same trials and label reuse, while allowing only one to replace its program, tests a more specific contribution of program revision.

3.5 What the implementation checks

The reference implementation provides a finite checker for unique event/artifact identifiers, visible inputs, version matching, protected criteria, target-specific permissions, and patch budgets. Constructed fixtures accept a valid trace and patch and reject a hidden input and five invalid patch variants. A separate target test allows a release owner to change routing but rejects the same actor’s audit-rule change; granting the review board that right admits it. This checks the operational consequences of the declared boundary.

The mechanism experiment instantiates a narrower executable subset: retained label statistics, retained trial scores, program versions, authorized patch application, label budgets, and a modeled cost ledger. It exports trajectory outcomes, program proportions, conditional block outcomes, and example version transitions. Full implementations would additionally need to enforce all relationships in [table 3](#), including criterion and retention contracts. The supplied checks establish conformance to their implemented predicates; the numerical experiment supplies evidence about efficacy.

Mapping public repository records supplies a complementary feasibility check. Three merged Flask PRs yield four commit artifacts and 56 mapped events: eight reviews, 42 head-commit check runs, and six opening/merge records. Artifact identifiers and recorded actions are recoverable. Historical authority, actual message exposure, effort, organization versions, and AI involvement remain unknown. [Appendix A](#) specifies the snapshot and mapping. Such a trace can expose instrumentation gaps before a team attempts a workflow-effect estimate.

4 Mechanism study: discovering, retaining, and updating evidence

4.1 Research questions and organizational instance

The experiment follows a change from its discovery to its later use. It asks whether evidence retention alters the value of evaluator revision, whether a one-time discovery can substitute for repeated search, and which part of reassessment produces a measured gain. These questions instantiate $U.a$ (evidence acquisition), M (retained evidence), and $U.r$ (continued use or reconsideration) in the specification.

The relations in [table 3](#) constrain specific design decisions. Decision–memory requires separate records for workflow labels, program scores, and the adopted program; we therefore intervene on these memories separately. Cost–resource requires each queried label to enter one ledger, even when both program trials and workflow selection reuse it. Actor–evidence requires current program trials to share the same pre-review information and prevents access to hidden error probabilities. Outcome–criterion keeps the deployment horizon and net-value rule common across arms. The experiment uses these obligations to distinguish a change in the evaluator from a change in its evidence.

A proposer supplies three workflow templates, an evaluator acquires labels, and a review board may replace the evaluation program. The roles denote information access and decision rights and can be assigned to humans or agents. The environment has two equally frequent observable task strata. Error probabilities are (.20, .20) for the standard template, (.16, .16) for broad, and initially (.08, .46) for specialized. Thus broad is initially best, while specialized helps one stratum and harms the population. In *stationary harm* these probabilities remain fixed. In *workflow reversal*, specialized becomes (.08, .08) before round 25 and becomes the best template. *Uniform gain* uses that improving specialized template throughout. The probabilities define synthetic mechanism contrasts, not fitted organizational parameters.

Each of 48 rounds deploys one template on 4096 production tasks. Only acquired evaluation labels enter decisions; the external scorer uses exact expected production outcomes. This separates uncertainty in organizational judgment from production sampling noise. One value unit is the gross value assigned to one completed production task before errors and costs. Net value per task subtracts error penalties and modeled expenses in these normalized units. An error costs 3 units, generation costs .02 per production or evaluation output, a label costs .20, and changing evaluation programs costs .50. Storage and computation costs are outside this ledger.

Table 4: Decision rules and the comparisons they support. All arms use the same memory condition and block-level resource ceiling.

Arm	Program decision	Role in the study
Biased / Balanced / Neyman	Program specified before the experiment	Fixed designs with different coverage/allocation rules.
Successive rejection	Fixed elimination rule across templates	A stronger selection-oriented comparator.
Discover once	Compare programs once; retain the winner	Discovery followed by reuse and budget reallocation.
Repeated discovery	Reassess every eight rounds	Incremental value of continued program assessment.
Trial-matched Balanced	Run the same trials; keep Balanced	Exploratory control for evidence acquisition and timing.

4.2 Memory and decision rules

The population-weighted posterior error estimate of template k is

$$\hat{r}_{k,t} = \frac{1}{2} \sum_{s=1}^2 \frac{1 + E_{ks,t}}{2 + N_{ks,t}}, \quad (5)$$

where E and N count observed errors and labels available to the decision. All conditions begin with Beta(1,1). *Reset* uses current-round evidence only. *Cumulative* retains all earlier evidence. *Window 8* retains evidence from the preceding eight rounds plus current observations. The same retention rule applies to past evaluation-program scores; a program’s identity remains retained until another is selected. This separates remembering a decision from remembering the evidence supporting it.

Three evaluation programs form a fixed catalog. Biased allocates 95% of a template’s allowance to the first stratum. Balanced uses equal counts. Neyman uses a two-label-per-cell pilot and posterior variance estimates to allocate the remainder across strata [38]. Each program chooses the template minimizing [equation \(5\)](#). Counts use population weights regardless of sampling proportions.

Workflow selection is a fixed-budget identification problem: evaluation outputs are acquired to choose a template for subsequent deployment. Successive Rejects addresses this objective by progressively eliminating candidates [5]; exploration sampling explicitly targets policy choice rather than immediate experimental outcomes [30]. Our additional successive-rejection comparator allocates across templates, whereas Neyman allocates within each template. It observes paired stratum labels, eliminates the worst estimated candidate after a first phase, and compares the remaining two after a second phase. Historical evidence enters the same estimates under the memory conditions; this is a memory-augmented implementation of the rejection schedule, with no claim to transfer stationary theoretical guarantees to the reversal environment. Allocation formulas and tie rules appear in [Appendix C](#).

4.3 Discovery, reuse, and fixed resource ceilings

[Table 4](#) distinguishes knowing an evaluator, discovering it once, and periodically reconsidering it. A label unit is one observed binary error outcome for one evaluated output. Every arm receives a ceiling of 2736 such observations per eight-round block. A fixed arm can allocate 342 units per round, or 114 per template for the three catalog programs. Discover once pays for program comparisons only before round 1. Repeated discovery compares programs before rounds 1, 9, 17, 25, 33, and 41. Both begin with Biased and select among the same catalog.

In the base design, program trials receive at most 20% of the block ceiling, with one trial per catalog program. Each trial screens the three workflows under the same pre-review memory snapshot, selects one, and validates it on fresh balanced labels. Its score is validation value less screening expense per production task. The highest mean retained trial score determines the program. This fixed outer selector implements bounded program revision. It does not generate new algorithms.

All acquired trial labels, including evidence from rejected programs, can enter workflow memory exactly once. Decisions about program scores precede pooling the current trials; the subsequent operational screen receives the pooled evidence. This makes failed searches potential sources of reusable information. Actual trial expenditure is deducted from the current block, and the remaining allowance is spread across its operational screens. Discover once receives the full operational allowance from block 2 onward. No arm borrows from a future block; unused units due to integer allocation are not charged.

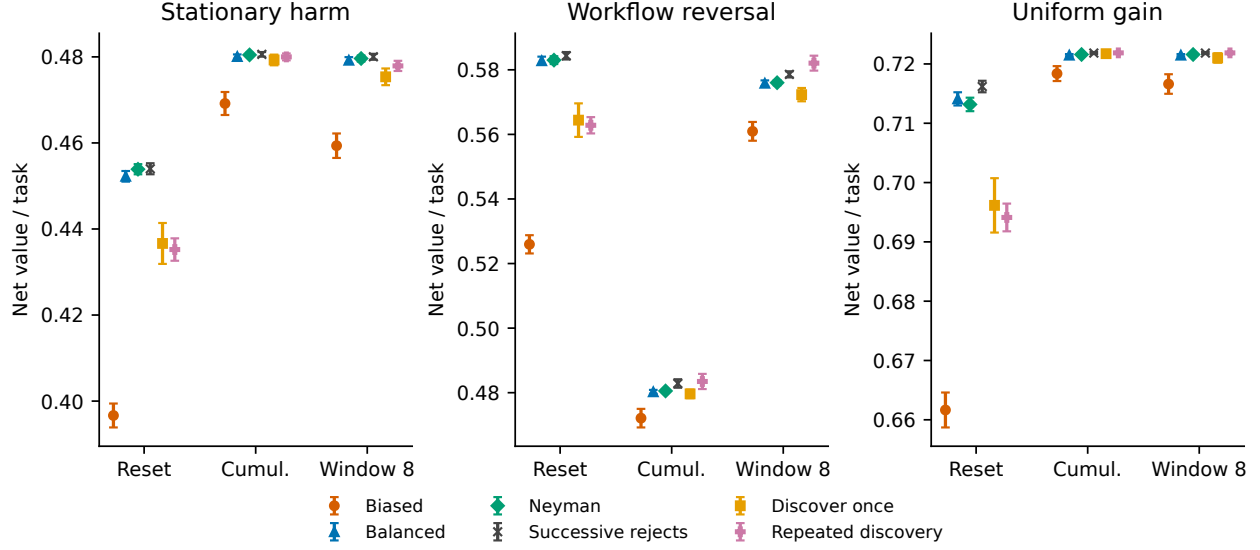


Figure 2: All core comparisons: 128 replicate trajectories per cell, means and 95% Monte Carlo intervals. Each panel uses its own value scale. “Cumul.” retains all acquired evidence; Window 8 retains the preceding eight rounds plus current evidence. The results concern the jointly specified memory, discovery, and acquisition rules.

Net value includes production generation, all acquired labels, all evaluated outputs, and program changes. Selection regret measures the gross value gap to the best available template, separately from those costs. Harmful adoption counts choices worse than standard; after reversal, this measure becomes zero for all templates, so regret and adaptation trajectories are needed to distinguish them. The primary outcome averages net value over all 48 rounds, with final-eight-round performance as a separate endpoint.

The core matrix crosses three environments, three memory rules, and six arms, using 128 independent replicate streams per cell. A separate 96-replicate grid crosses program-budget shares .2, .4, .6 with trial counts 1, 3, 6, keeping the total ceiling fixed. It also removes program-score memory or trial-label pooling separately. After inspecting the core results, we added an explicitly exploratory trial-matched control and reversals before rounds 21 and 29 on new streams, to test attribution and timing. All conditions and outputs are retained in Appendix C and the reproducibility package.

4.4 Accumulation changes the apparent cost of improvement

Figure 2 reports every core arm and memory condition. Under stationary harm, cumulative evidence raises Balanced’s mean net value from .45224 to .48007, a paired gain of .02783 (95% Monte Carlo interval .02659–.02907). Its final-eight-round value reaches .48163, the value of consistently selecting broad under its actual evaluation expenditure. Balanced’s mean selection regret falls from .02939 to .00156 per task, while its evaluation expense remains .01837. The net-value gain therefore equals the reduction in selection regret.

Repeated discovery is .01702 below Balanced with reset evidence (interval $-.01964$ to $-.01440$). With cumulative evidence, that difference becomes $-.00007$ (interval $-.00100$ to $.00085$). Discover once reaches .47928, and repeated discovery .48000. Thus the sizeable reset disadvantage does not persist when paid-for evidence is reused. This does not establish equivalence: the experiment estimates a small difference with the stated uncertainty. Successive rejection reaches .48059 under cumulative evidence, providing a competitive fixed selection rule.

The comparison to fixed programs also depends on prior knowledge. An equal-probability initial draw from the three catalog programs defines an ex ante fixed-program mixture, computed as their average conditional value. Under stationary harm, its net values are .43426 with reset evidence and .47657 with cumulative evidence. Repeated discovery exceeds these prior-averaged references by .00096 and .00343, respectively. Table 6 reports the mixture in every core condition. Balanced is a prespecified representative-sampling rule, not an ex post oracle. These comparisons distinguish the cost of discovering a program from the value of revising an already reasonable design.

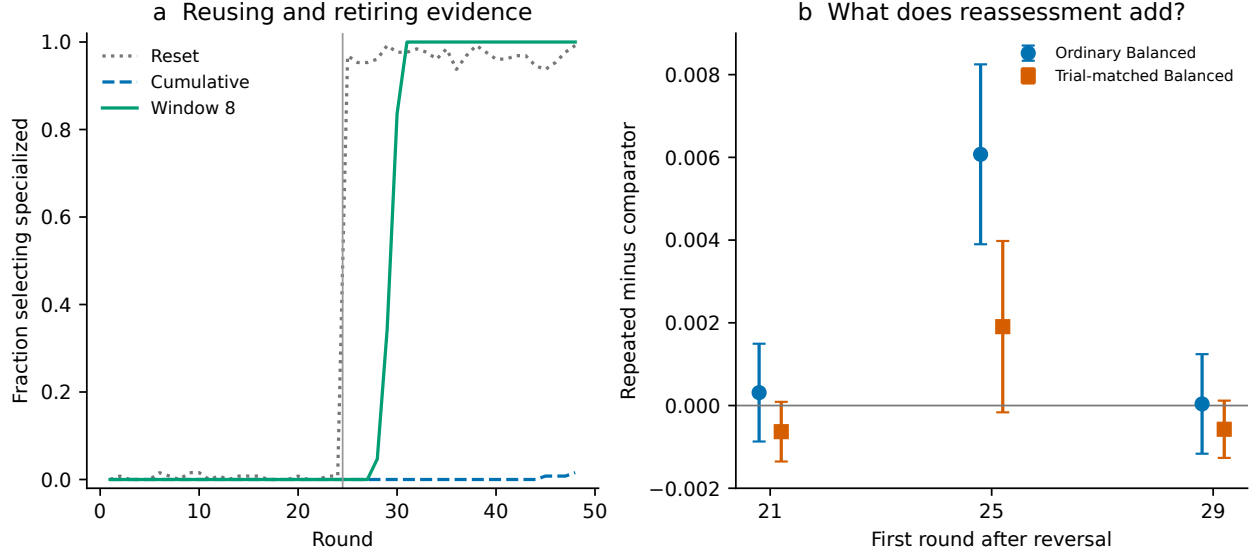


Figure 3: Retention and attribution. (a) Fixed Balanced under the core reversal; the vertical line marks the change before round 25. (b) Exploratory controls with Window 8 and reversals before rounds 21, 25, or 29. Points show repeated-discovery minus comparator differences with paired 95% Monte Carlo intervals on 128 fresh replicate streams. Matching the trials, label reuse, and timing changes the attribution of the apparent reassessment gain.

4.5 Evidence can become obsolete before an organization forgets it

Under workflow reversal, indefinite accumulation slows the replacement of broad by the newly superior specialized template. Balanced’s full-horizon values are .48026 with cumulative memory, .57583 with Window 8, and .58280 with reset evidence. The finite window improves considerably on unlimited accumulation, yet its transition delay makes it worse than reset over this particular horizon. In the final eight rounds it reaches .72163, compared with .71237 for reset and .48280 for cumulative memory. Figure 3a displays the corresponding deployment choices.

With Window 8 in the core reversal, repeated discovery exceeds discover-once by .00975 (interval .00709–.01242) and ordinary Balanced by .00623. These differences include the effects of concentrated trial acquisition and pooling, as well as the possibility of replacing the program. On the fresh exploratory streams, reversal before round 25 gives a repeated-discovery advantage of .00607 over ordinary Balanced (interval .00390–.00825). Matching program trials and label reuse while keeping Balanced reduces this difference to .00191 (interval –.00016 to .00398). For reversals before rounds 21 and 29, repeated discovery minus trial-matched Balanced is –.00063 and –.00057, respectively; both intervals span zero. The corresponding differences against ordinary Balanced are .00031 and .00004. The design therefore does not identify a stable additional gain from program replacement across the tested timings.

4.6 What the allocation and retention diagnostics explain

The full nine-cell budget-share/trial-count grid is reported in table 7. More trial expenditure can improve the total repeated-discovery package when it also provides reusable workflow evidence. Consequently, “meta-evaluation expenditure” is not synonymous with discarded operational evidence. Removing trial-label pooling lowers repeated discovery from .48043 to .47751 under stationary cumulative memory, and from .58214 to .57392 under reversal with Window 8 on the independent sensitivity streams. Resetting only the program-score history has much smaller mean effects in these two conditions. These are interventions on different memory components, rather than interchangeable definitions of learning.

Per-review program proportions and subsequent-block outcomes conditional on the retained program are supplied in table 8 and results/learning_conditional.csv. At the last stationary review with cumulative memory, Biased, Balanced, and Neyman are retained in 38.3%, 31.2%, and 30.5% of runs. The 40 runs retaining Balanced have subsequent-block net value .4819 and zero harmful adoption. Accurate workflow selection therefore need not coincide with convergence to one program when programs share a well-informed workflow memory. These summaries describe selection and downstream performance together; because retained programs are selected endogenously, their conditional means are not causal program effects. The combined evidence supports a narrower and more useful

explanation of organizational improvement: gains depend on what the organization learns from evaluation, how long that evidence remains relevant, and whether a measured gain survives a control with the same information acquisition.

5 Implications for human–agent organizations

5.1 Comparing capability and organizational change

The specification makes an otherwise ambiguous comparison operational. A capability-only intervention changes actor profiles while keeping assignment, visibility, authority, and improvement procedures fixed. A workflow intervention changes those arrangements under a fixed procedure. A procedure intervention changes how candidate arrangements are evaluated or retained. A factorial study can compare these interventions and their interactions under the same task population and outcome criterion. The bottleneck example in [section 2](#) supplies one elementary prediction; the mechanism study shows how the retention and acquisition of evidence alter the value of organizational search.

The distinction matters when AI changes both production volume and the work of supervision. Automation can redistribute monitoring demands [6]; human and machine learning can become organizationally interdependent [45]. In the experiment, the same amount of acquired evidence has different value depending on whether the organization reuses it, discards it, or retains it after the relevant task relationship changes. The conditional implication is to evaluate evidence retention together with the workflow that generates evidence.

The supplementary audit model in [Appendix D](#) examines the earlier decision to acquire information at all. Its finite-horizon value-of-information controller can acquire little evidence under a strong prior, while the main study asks what happens to evidence after it has been bought. Together the models distinguish acquisition, retention, and use. Neither an audit count nor a revision count alone measures an organization’s capacity to learn.

5.2 Independence, interaction order, and cross-functional breadth

Independent agent identities do not establish independent evidence. Shared models, training data, tools, or copied explanations can correlate errors, limiting the value of aggregation [16]. Social influence can also change the diversity of judgments [34]. An ROI comparison records both pre-existing covariance assumptions and reveal events. It can then vary model diversity, information access, or communication while keeping the decision rule explicit.

Human-first and agent-first interaction differ through the information available when a person commits an assessment. Private commitment can preserve an additional signal, while early advice can supply knowledge the person lacks. The outcome depends on capability, covariance, reliance, and the aggregation rule. The elementary calculation in [Appendix B](#) shows that apparent gains from exposure can equal a reweighting of private evidence. Human–AI studies of reliance and cognitive forcing motivate additional behavioral measurements [7, 12]; reproducing their effects requires their task and participant conditions, not just matching a curve with selected parameters.

Cross-functional humans can reduce handoffs, interpret errors across domains, and connect agent outputs to downstream requirements. Organization design and coordination requirements identify the dependencies through which these gains could arise [14, 20, 41]. The gain must be compared with lost specialist depth, attention limits, and switching costs. A useful experiment varies human breadth while logging transfer count, rework, review delay, and quality at fixed resources. These variables are representable in the specification, while their empirical magnitudes remain task-dependent.

5.3 Weak-to-strong evaluation and more capable agents

Weak-to-strong generalization concerns learning under weaker supervision [13]. In a human–agent organization, a related difficulty appears when reviewers cannot directly assess the outputs or proposed process changes of a more capable agent. Execution capability, evaluation reliability, and authority then become separate design dimensions. The organization can invest in decomposed tests, independent evidence, selective escalation, or stronger evaluation tools; each choice changes the cost and coverage of its improvement procedure. Work on amplified supervision and scalable oversight provides candidate mechanisms for this setting [9, 15].

Hypothetical artificial superintelligence would widen the gap between execution and human evaluation capability. The same specification would require independently measurable outcomes and explicit authority for revising evaluation procedures.

5.4 Validity and next empirical test

The specification is evaluated through a reference checker, a public-data mapping, and a controlled mechanism instance. This establishes a path from named modeling fields to executable decisions and observable omissions. It does

not yet establish lower modeling effort, better inter-rater agreement, or better predictions than alternative modeling notations. Those outcomes require users to model the same organizations under competing specifications.

For the mechanism, the catalog, known stratum weights, synthetic error distributions, and cost schedule define the comparison. Program trials evaluate a fixed catalog at a smaller screening budget than operational deployment. Historical program scores average trials conducted at different knowledge states; they are a heuristic selection rule, not a stationary posterior for program quality. Reset discards reusable evidence; cumulative retention assumes past outcomes remain relevant; the fixed window bounds age without detecting a change. These choices explain the limits of the tested learning rules. Concept-drift, switching-bandit, and change-point methods offer principled alternatives for future instances [1, 21, 22], with acquisition costs represented explicitly [46].

The acquisition-matched control shows why an improved organizational outcome should not be attributed to the most conspicuous changed component. Periodic program trials also alter when evidence arrives and who can reuse it. The observed gains do not establish that program replacement supplies a stable additional benefit across the tested change timings. A positive result for open-ended procedure discovery would require a broader candidate space and a comparator with equally informative observations.

A direct organizational test would randomize review-routing changes and evaluation coverage across comparable task streams, retain failed proposals, and measure escaped errors and human attention over a declared horizon. A second stage could randomize whether teams may replace the evaluation program. Such a design would separate the value of broader evidence from the value of the procedure used to discover and retain that evidence policy.

6 Conclusion

Recursive organization improvement asks how human–agent teams change their work and learn whether those changes helped. The proposed modeling specification connects evidence access, organizational memory, authority, costs, and retention through explicit change contracts. Its mechanism study shows why these connections matter: accumulated evidence largely removes the reset design’s penalty for repeated assessment in a stable environment, while obsolete evidence can delay a necessary workflow change. Matching acquisition and reuse then narrows the gain attributable to replacing the evaluator itself. Organizational improvement must therefore be evaluated across the lifecycle of its evidence—what is discovered, what is retained, and when it is reconsidered—rather than inferred from agent capability or the number of revisions performed.

Code and data availability

Code and data are available at <https://github.com/wizardlancet/recursive-organization-improvement>. The tagged release `research-2026-09-30` contains the reference checker, simulation and analysis code, frozen protocols, per-trajectory results, and a pseudonymized public-record snapshot. Original code is licensed under MIT; the manuscript, original figures, and synthetic data are licensed under CC BY 4.0. Third-party materials retain their original terms. Reproduction commands and checksums are supplied in the repository.

Use of AI tools

OpenAI Codex assisted with manuscript drafting and revision, simulation implementation, data analysis, and figure preparation. The reported mechanism experiments use synthetic label distributions; they do not measure deployed language-model performance.

References

- [1] Ryan Prescott Adams and David J. C. MacKay. Bayesian online changepoint detection, 2007. URL <https://arxiv.org/abs/0710.3742>.
- [2] Agile Alliance. Extreme Programming (XP), n.d. URL <https://agilealliance.org/glossary/xp/>. Undated living glossary; accessed 2026-09-28.
- [3] Gene M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In *Proceedings of the April 18-20, 1967, spring joint computer conference on - AFIPS '67 (Spring)*, pages 483–485, 1967. doi: 10.1145/1465482.1465560.
- [4] Chris Argyris and Donald A. Schön. *Organizational Learning: A Theory of Action Perspective*. Addison-Wesley, 1978. URL <https://search.worldcat.org/title/610749704>.
- [5] Jean-Yves Audibert, Sébastien Bubeck, and Rémi Munos. Best arm identification in multi-armed bandits. In *Proceedings of the 23rd Conference on Learning Theory*, pages 41–53, 2010. URL https://www.microsoft.com/en-us/research/wp-content/uploads/2017/01/COLT10_ABM.pdf.

- [6] Lisanne Bainbridge. Ironies of automation. *Automatica*, 19(6):775–779, 1983. doi: 10.1016/0005-1098(83)90046-8.
- [7] Gagan Bansal, Tongshuang Wu, Joyce Zhou, Raymond Fok, Besmira Nushi, Ece Kamar, Marco Tulio Ribeiro, and Daniel Weld. Does the Whole Exceed its Parts? The Effect of AI Explanations on Complementary Team Performance. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, pages 1–16, 2021. doi: 10.1145/3411764.3445717.
- [8] Kent Beck, Mike Beedle, Arie van Bennekum, Alistair Cockburn, Ward Cunningham, Martin Fowler, James Grenning, Jim Highsmith, Andrew Hunt, Ron Jeffries, Jon Kern, Brian Marick, Robert C. Martin, Steve Mellor, Ken Schwaber, Jeff Sutherland, and Dave Thomas. Principles behind the Agile Manifesto, 2001. URL <https://agilemanifesto.org/principles.html>. Accessed 2026-09-28.
- [9] Samuel R. Bowman, Jeeyoon Hyun, Ethan Perez, Edwin Chen, Craig Pettit, Scott Heiner, Kamilë Lukošūūtė, Amanda Askell, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, Christopher Olah, Daniela Amodei, Dario Amodei, Dawn Drain, Dustin Li, Eli Tran-Johnson, Jackson Kernion, Jamie Kerr, Jared Mueller, Jeffrey Ladish, Joshua Landau, Kamal Ndousse, Liane Lovitt, Nelson Elhage, Nicholas Schiefer, Nicholas Joseph, Noemí Mercado, Nova DasSarma, Robin Larson, Sam McCandlish, Sandipan Kundu, Scott Johnston, Shauna Kravec, Sheer El Showk, Stanislav Fort, Timothy Telleen-Lawton, Tom Brown, Tom Henighan, Tristan Hume, Yuntao Bai, Zac Hatfield-Dodds, Ben Mann, and Jared Kaplan. Measuring Progress on Scalable Oversight for Large Language Models, 2022. URL <https://arxiv.org/abs/2211.03540>; preprint.
- [10] Erik Brynjolfsson, Daniel Rock, and Chad Syverson. The Productivity J-Curve: How Intangibles Complement General Purpose Technologies. *American Economic Journal: Macroeconomics*, 13(1):333–372, 2021. doi: 10.1257/mac.20180386.
- [11] Erik Brynjolfsson, Danielle Li, and Lindsey Raymond. Generative AI at Work. *The Quarterly Journal of Economics*, 140(2): 889–942, 2025. doi: 10.1093/qje/qjae044.
- [12] Zana Bućinca, Maja Barbara Malaya, and Krzysztof Z. Gajos. To Trust or to Think: Cognitive Forcing Functions Can Reduce Overreliance on AI in AI-assisted Decision-making. *Proceedings of the ACM on Human-Computer Interaction*, 5(CSCW1): 1–21, 2021. doi: 10.1145/3449287.
- [13] Collin Burns, Pavel Izmailov, Jan Hendrik Kirchner, Bowen Baker, Leo Gao, Leopold Aschenbrenner, Yining Chen, Adrien Ecoffet, Manas Joglekar, Jan Leike, Ilya Sutskever, and Jeffrey Wu. Weak-to-strong generalization: Eliciting strong capabilities with weak supervision. In Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp, editors, *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 4971–5012. PMLR, 21–27 Jul 2024. URL <https://proceedings.mlr.press/v235/burns24b.html>.
- [14] Marcelo Cataldo, Patrick A. Wagstrom, James D. Herbsleb, and Kathleen M. Carley. Identification of coordination requirements: Implications for the design of collaboration and awareness tools. In *Proceedings of the 2006 20th anniversary conference on Computer supported cooperative work*, pages 353–362, 2006. doi: 10.1145/1180875.1180929.
- [15] Paul Christiano, Buck Shlegeris, and Dario Amodei. Supervising strong learners by amplifying weak experts, 2018. URL <https://arxiv.org/abs/1810.08575>. arXiv:1810.08575; preprint.
- [16] Robert T. Clemen and Robert L. Winkler. Limits for the precision and value of information from dependent sources. *Operations Research*, 33(2):427–442, 1985. doi: 10.1287/opre.33.2.427.
- [17] Virginia Dignum, Frank Dignum, and John-Jules Meyer. An agent-mediated approach to the support of knowledge sharing in organizations. *The Knowledge Engineering Review*, 19(2):147–174, 2004. doi: 10.1017/s0269888904000244.
- [18] Douglas C. Engelbart. Toward High-Performance Organizations: A Strategic Role for Groupware. In *Proceedings of GroupWare '92*, 1992. URL <https://www.dougenelbart.org/pubs/papers/scanned-original/1992-augment-132811-Toward-High-Performance-Organizations-a-Strategic-Role-for-Groupware.pdf>.
- [19] Martha S. Feldman and Brian T. Pentland. Reconceptualizing organizational routines as a source of flexibility and change. *Administrative Science Quarterly*, 48(1):94–118, 2003. doi: 10.2307/3556620.
- [20] Jay R. Galbraith. Organization Design: An Information Processing View. *Interfaces*, 4(3):28–36, 1974. doi: 10.1287/inte.4.3.28.
- [21] João Gama, Indrë Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4):1–37, 2014. doi: 10.1145/2523813.
- [22] Aurélien Garivier and Eric Moulines. On upper-confidence bound policies for switching bandit problems. In *Algorithmic Learning Theory*, volume 6925 of *Lecture Notes in Computer Science*, pages 174–188, 2011. doi: 10.1007/978-3-642-24412-4_16.
- [23] GitHub. REST API endpoints for check runs. GitHub REST API documentation, 2026. URL <https://docs.github.com/en/rest/checks/runs>. Accessed September 30, 2026.
- [24] GitHub. REST API endpoints for pull request reviews. GitHub REST API documentation, 2026. URL <https://docs.github.com/en/rest/pulls/reviews>. Accessed September 30, 2026.
- [25] D. G. Horvitz and D. J. Thompson. A Generalization of Sampling Without Replacement from a Finite Universe. *Journal of the American Statistical Association*, 47(260):663–685, 1952. doi: 10.1080/01621459.1952.10483446.
- [26] Eric Horvitz. Principles of mixed-initiative user interfaces. In *Proceedings of the SIGCHI conference on Human factors in computing systems the CHI is the limit - CHI '99*, pages 159–166, 1999. doi: 10.1145/302979.303030.
- [27] Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. In *International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/file/36b7acf6f6010652

- [b3f2a433774a66fe-Paper-Conference.pdf](#).
- [28] Jomi Fred Hübner, Jaime Simão Sichman, and Olivier Boissier. MOISE+: towards a structural, functional, and deontic model for MAS organization. In *Proceedings of the first international joint conference on Autonomous agents and multiagent systems: part I*, pages 501–502, 2002. doi: 10.1145/544741.544858.
 - [29] Matthew Johnson, Jeffrey M. Bradshaw, Paul J. Feltovich, Catholijn M. Jonker, M. Birna Van Riemsdijk, and Maarten Sierhuis. Coactive Design: Designing Support for Interdependence in Joint Activity. *Journal of Human-Robot Interaction*, 3(1):43–69, 2014. doi: 10.5898/jhri.3.1.johnson.
 - [30] Maximilian Kasy and Anja Sautmann. Adaptive treatment assignment in experiments for policy choice. *Econometrica*, 89(1): 113–132, 2021. doi: 10.3982/ECTA17527. URL <https://doi.org/10.3982/ECTA17527>.
 - [31] J. D. Lee and K. A. See. Trust in Automation: Designing for Appropriate Reliance. *Human Factors: The Journal of the Human Factors and Ergonomics Society*, 46(1):50–80, 2004. doi: 10.1518/hfes.46.1.50_30392.
 - [32] Barbara Levitt and James G. March. Organizational Learning. *Annual Review of Sociology*, 14(1):319–338, 1988. doi: 10.1146/annurev.so.14.080188.001535.
 - [33] John D. C. Little. A Proof for the Queuing Formula: $L = \lambda W$. *Operations Research*, 9(3):383–387, 1961. doi: 10.1287/opre.9.3.383.
 - [34] Jan Lorenz, Heiko Rahut, Frank Schweitzer, and Dirk Helbing. How social influence can undermine the wisdom of crowd effect. *Proceedings of the National Academy of Sciences*, 108(22):9020–9025, 2011. doi: 10.1073/pnas.1008636108.
 - [35] Thomas W. Malone and Kevin Crowston. The interdisciplinary study of coordination. *ACM Computing Surveys*, 26(1): 87–119, 1994. doi: 10.1145/174666.174668.
 - [36] James G. March. Exploration and Exploitation in Organizational Learning. *Organization Science*, 2(1):71–87, 1991. doi: 10.1287/orsc.2.1.71.
 - [37] Paul Milgrom and John Roberts. Complementarities and fit: Strategy, structure, and organizational change in manufacturing. *Journal of Accounting and Economics*, 19(2-3):179–208, 1995. doi: 10.1016/0165-4101(94)00382-F.
 - [38] Jerzy Neyman. On the two different aspects of the representative method: The method of stratified sampling and the method of purposive selection. *Journal of the Royal Statistical Society*, 97(4):558–606, 1934. doi: 10.1111/j.2397-2335.1934.tb04184.x.
 - [39] Shakked Noy and Whitney Zhang. Experimental evidence on the productivity effects of generative artificial intelligence. *Science*, 381(6654):187–192, 2023. doi: 10.1126/science.adh2586.
 - [40] R. Parasuraman, T.B. Sheridan, and C.D. Wickens. A model for types and levels of human interaction with automation. *IEEE Transactions on Systems, Man, and Cybernetics - Part A: Systems and Humans*, 30(3):286–297, 2000. doi: 10.1109/3468.844354.
 - [41] Phanish Puranam. Human–AI collaborative decision-making as an organization design problem. *Journal of Organization Design*, 10(2):75–80, 2021. doi: 10.1007/s41469-021-00095-2.
 - [42] Phanish Puranam, Oliver Alexy, and Markus Reitzig. What’s “New” About New Forms of Organizing? *Academy of Management Review*, 39(2):162–180, 2014. doi: 10.5465/amr.2011.0436.
 - [43] Winston W. Royce. Managing the Development of Large Software Systems. In *Proceedings of IEEE WESCON*, pages 1–9, 1970. URL <https://cse.msu.edu/~cse435/Homework/HW3/royce1970.pdf>. Original 1970 paper; linked copy is a reprint.
 - [44] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html.
 - [45] Timo Sturm, Jin P. Gerlach, Luisa Pumplun, Neda Mesbah, Felix Peters, Christoph Tauchert, Ning Nan, and Peter Buxmann. Coordinating Human and Machine Learning for Effective Organizational Learning. *MIS Quarterly*, 45(3):1581–1602, 2021. doi: 10.25300/misq/2021/16543.
 - [46] Aaron D. Tucker, Caleb Biddulph, Claire Wang, and Thorsten Joachims. Bandits with costly reward observations. In *Proceedings of the Thirty-Ninth Conference on Uncertainty in Artificial Intelligence*, volume 216 of *Proceedings of Machine Learning Research*, pages 2147–2156, 2023. URL <https://proceedings.mlr.press/v216/tucker23a.html>.
 - [47] Michelle Vaccaro, Abdullah Almaatouq, and Thomas Malone. When combinations of humans and AI are useful: A systematic review and meta-analysis. *Nature Human Behaviour*, 8(12):2293–2303, 2024. doi: 10.1038/s41562-024-02024-1.
 - [48] Wil van der Aalst. Process mining: Overview and opportunities. *ACM Transactions on Management Information Systems*, 3(2):1–17, 2012. doi: 10.1145/2229156.2229157.
 - [49] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. Aflow: Automating agentic workflow generation. In *International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/5492ecbce4439401798dcd2c90be94cd-Abstract-Conference.html.
 - [50] Maurizio Zollo and Sidney G. Winter. Deliberate Learning and the Evolution of Dynamic Capabilities. *Organization Science*, 13(3):339–351, 2002. doi: 10.1287/orsc.13.3.339.2780.

A Instantiating the specification: a three-PR feasibility mapping

An instance begins by declaring its population, criterion, horizon, and change boundary. It then assigns actors to roles, defines dependencies and visibility, and supplies the initial arrangement and improvement procedure. Evidence references identify versioned artifacts rather than undifferentiated transcript text. Each patch records the state it expects and the evidence its proposer could inspect. A compact contract can use the following interchange form; the referenced artifacts contain the population, protocol, scores, and retention details.

```
{
  "id": "coverage-change-1",
  "target": "evaluation.coverage_program",
  "expected_version": 0,
  "actor": "review_board",
  "transformation": {"from": "Biased", "to": "Balanced"},
  "evidence": ["program-trials-1.v1"],
  "comparison": "population-and-net-value-criterion.v1",
  "cost_ledger": "labels-outputs-transition-1.v1",
  "horizon_rounds": 8,
  "retention": "next-review-rule.v1"
}
```

This example is a proposed full contract. The executable Patch class implements target, version, actor, evidence, reversibility, and admission cost; other contract fields are supplied by the surrounding experiment protocol. Reversibility is recorded but the checker does not simulate rollback. A diagnostic coverage trace contains per-round sample counts, estimates, decisions, cost totals, and authorized program changes. The outcome equations reconstruct net value from the selected workflow and ledger, while negative fixtures exercise admission failures.

The public mapping uses a frozen convenience sample from `pallets/flask`: the first three merged PRs before September 30, 2026 among the first 100 closed PRs returned in descending creation order. The initial retrieval was September 28, 2026 at 17:45:43 UTC (September 29 in Singapore). Selection candidates, URLs, timestamps, and response hashes are retained. The snapshot contains factual identifiers and states, excluding message bodies and email addresses. Released actor identifiers are stable pseudonyms; account IDs and usernames are omitted. PR links remain available for provenance, so this is pseudonymization rather than irreversible anonymization.

For each PR the adapter retrieves commits, reviews, and checks at its recorded head SHA, checking pagination for the subordinate lists. Table 5 summarizes the mapped records. Commits are artifact versions; opening, review, check, and merge records are events. Endpoint definitions support these mappings [23, 24]. Earlier heads, test-merge commits, off-platform activity, and historical branch protection require additional sources. Historical permission and actual evidence

Table 5: Frozen public-record mapping. These counts describe record availability, not organizational improvement or AI participation.

PR	Commit artifacts	Reviews	Head checks	Events
#6133	1	0	14	16
#6096	2	5	14	21
#6095	1	3	14	19

exposure remain unknown even when a merge and a public review are visible. The adapter preserves these unknown fields and does not pass the records to the complete-trace validator. No workflow intervention or counterfactual is observed. A generic log augmented with the same missing information would support the same checks; the practical benefit tested here is identifying the required connections and missing instrumentation.

B Analytical background for interaction and evaluation

B.1 Coverage and indistinguishability

Suppose two environments induce identical distributions over every history accessible to a procedure, but a specified workflow change improves the population in one and harms it in the other. Any binary sign decision based on those histories has the same output distribution in both environments. If its probability of reporting improvement is q , its two error probabilities are $1 - q$ and q . Their sum is one, so at least one is at least $1/2$. This is the standard observation-equivalence argument.

A construction uses equally frequent flagged and unflagged cases. A change lowers flagged error from .20 to .10 in both environments; unflagged error falls to .10 in one and rises to .50 in the other. If only flagged labels are observed and no downstream signal arrives, evidence is identical although population risk changes from .20 to .10 or .30. With known positive inclusion probabilities q_i and accurate labels E_i , the Horvitz–Thompson estimator $N^{-1} \sum_i Z_i E_i / q_i$ is design-unbiased for finite-population risk [25]. Small inclusion probabilities can still produce high variance. The main experiment has positive coverage in every cell and tests this finite-evidence problem; it is not an instance of exact observational equivalence.

B.2 Dependent evidence and interaction order

For equal-variance errors with equicorrelation ρ , the variance of the mean is $\sigma^2[\rho + (1 - \rho)/n]$, with $-1/(n - 1) \leq \rho \leq 1$. Common bias contributes an additional squared-bias term to mean squared error. Information dependence therefore limits the gain from adding actors [16]; more actors alone need not produce proportionally more information.

Let independent initial human and agent estimation errors have variances v_H and v_A . Suppose the human sees the agent answer and replaces the private judgment by $(1 - \alpha)H + \alpha A$. Equal averaging of that judgment with A has risk

$$R(\alpha) = \{(1 - \alpha)^2 v_H + (1 + \alpha)^2 v_A\} / 4. \quad (6)$$

For $v_A < v_H$ and $0 \leq \alpha \leq 1$, the optimum is $\alpha^* = (v_H - v_A) / (v_H + v_A)$, giving $v_H v_A / (v_H + v_A)$. This is also the risk of optimal inverse-variance weighting of the original private judgments. Thus a gain over equal private averaging is a reweighting gain, not evidence that exposure creates information. With initial covariance σ_{HA} , risk gains the term $(1 - \alpha^2)\sigma_{HA}/2$. The parameters describe a candidate interaction model; they are not empirical estimates of anchoring.

Cross-functional breadth can be represented by reduced transfer cost and changed capability profiles. A simple accounting decomposition is saved handoff cost plus improved integration, minus lost specialist depth and switching cost. Its sign depends on measured magnitudes and on the dependency graph. The specification preserves these separate terms instead of assigning a universal benefit to generalists.

C Learning experiment: complete specification and reporting

C.1 Frozen core design and exploratory controls

The local core protocol, `examples/learning_protocol.json`, was recorded before the first run of the core implementation. It specifies 48 rounds, a 2736-label ceiling for each eight-round block, three catalog programs, six arms, three memory conditions, three environments, and the full allocation grid. There was no parameter selection on test results. The trial-matched control and change-timing checks were specified after the core results exposed a possible acquisition-timing explanation; their separate protocol is `examples/learning_controls.json`. They use fresh streams and are reported as exploratory attribution checks. Neither protocol is an external preregistration.

The core has $3 \times 3 \times 6 \times 128 = 6912$ trajectories. The allocation grid uses 96 replicates for each share/trial-count combination and each discovery arm under stationary cumulative memory and reversal with Window 8. Four fixed references and two additional memory ablations in each environment bring this analysis to 4608 trajectories. The exploratory controls cross four environments (stationary and reversals before rounds 21, 25, 29), three memory conditions, four arms, and 128 replicates: 6144 trajectories. The three independent keyed root seeds are 920000, 930000, and 940000. Each random stream is indexed by root seed, round, acquisition stage, program, trial, and replicate. Conditions share underlying potential-label streams within each root; no policy receives unqueried labels.

C.2 Evidence update and program trials

For a template–stratum cell, cumulative memory adds each actually queried label once. Window 8 uses observations at historical rounds $t - 8$ through $t - 1$, plus current observations. Reset retains no earlier observations. All program trials at a review receive the same pre-review memory snapshot. Trial outputs and fresh validation labels are pooled only after program selection, then supplied to the current operational screen. Validation observes only the selected template, while screening observes all templates except for the second phase of the rejection comparator. A label from a rejected program remains usable if its source and population are appropriate. The pooling ablation withholds all program-trial labels from workflow memory while retaining program scores.

Each trial score is $1 - 3\hat{r}^{\text{val}} - .22N^{\text{screen}}/4096$, where validation uses equal counts across the two strata. A program’s score is the mean of its trial means retained under the memory rule. Each review has the same trial count within a configuration. Ties favor Biased, then Balanced, then Neyman. Program-score memory can be disabled independently of workflow memory. Retained scores were generated under different historical knowledge states; they are empirical assessments used by this specified controller, not identically distributed observations of an invariant program value.

For block budget $B = 2736$, program share f , three programs, and q trials per program, a trial receives $b = \lfloor \lfloor fB \rfloor / (3q) \rfloor$ label units. Validation obtains $v = \max\{1, \lfloor \lfloor .2b \rfloor / 2 \rfloor\}$ labels per stratum; screening receives $b - 2v$ total units. The base $(f, q) = (.2, 1)$ therefore allows 182 units per trial, with 18 validation labels per stratum and 146 units for screening. Actual counts can be smaller after integer allocation. Let M_i be the actual number of trial labels for replicate i at a block's start. Its subsequent per-round operational allowance is $\lfloor (B - M_i) / 8 \rfloor$. Discover once has $M_i = 0$ after block 1. This rule keeps the budget fixed when trial count or program share changes.

C.3 Within-template and between-template allocation

For the catalog programs, let u be one template's share of the operational or trial allowance. Balanced obtains $\lfloor u/2 \rfloor$ labels in each stratum. Biased obtains $\max\{1, \lfloor .95u \rfloor\}$ and $\max\{1, \lfloor .05u \rfloor\}$. All tested allocations fit the allowance. Neyman first obtains $m = \min\{2, \max\{1, \lfloor u/4 \rfloor\}\}$ labels per cell. Using retained and pilot data, it estimates p_s , sets $v_s = p_s(1 - p_s)$, and allocates

$$n_s = m + \left\lceil \frac{(u - 2m)\sqrt{v_s}}{\sum_{\ell} \sqrt{v_{\ell}}} \right\rceil. \quad (7)$$

Pilot labels enter the posterior once. Equal label costs make this the equal-cost stratified allocation. Catalog selection breaks posterior ties in template order standard, specialized, broad.

The successive-rejection comparator treats one pair of stratum labels as a bounded loss with mean equal to population risk. With n affordable pairs and $K = 3$, it uses the schedule of Audibert et al. [5]:

$$\overline{\log K} = \frac{1}{2} + \sum_{j=2}^K \frac{1}{j}, \quad n_k = \left\lceil \frac{n - K}{(\overline{\log K})(K + 1 - k)} \right\rceil, \quad k = 1, 2. \quad (8)$$

All three templates receive n_1 pairs; the worst posterior-mean template is removed, and the remaining two reach n_2 pairs each. Elimination ties remove the lowest indexed template; final ties use the lowest surviving index. Historical evidence enters the posterior under the memory conditions. Thus the implementation uses the published rejection schedule with declared posterior ranking and deterministic ties. At the fixed allowance of 342 labels it uses 336 labels. No theoretical bound is claimed for this modification under memory or drift. SR is an additional fixed comparator and is not part of the three-program discovery catalog.

C.4 Outcome accounting, checks, and uncertainty

For deployed template k_t with true population risk $r_{k_t, t}$, total queried labels Q_t , and program-change indicator S_t , the external score is

$$V_t = 1 - 3r_{k_t, t} - .02 - \{.22Q_t + .50S_t\} / 4096. \quad (9)$$

Each queried label corresponds to a generated evaluation output, so .22 includes .20 labeling and .02 generation. The separate .02 term prices each production output. Unqueried potential-label arrays are a simulator device, not actor-visible generated outputs. All rejected-program trials are charged. Program comparison costs are not subtracted a second time after computing net value.

The simulator checks every block ceiling and the identity $V_t = 1 - 3r_t^* - .02 - \text{regret}_t - \text{expense}_t$, where r_t^* is the best available risk. An independent scalar reconstruction checks 36 fixed-Balanced trajectories across the three environments and memory conditions to absolute tolerance 10^{-12} . Additional fixtures check the lookback boundary, rejection allocation, and selection of a known best template. The patch checker enforces target authority and state version for program changes.

Reported half-widths are $1.96s/\sqrt{N}$ across independent replicate trajectories. Paired intervals use within-replicate differences under the common streams. They quantify simulation uncertainty conditional on the supplied parameters. The uniform fixed benchmark averages the conditional mean values of Biased, Balanced, and Neyman under an initial uniform program prior; its paired contrast integrates over this discrete prior rather than adding random program draws. No claim of equivalence or correction for multiple hypothesis testing is attached to intervals spanning zero.

Table 6 supplies all core cell means; raw trajectories, intervals, costs, harms, regret, and late outcomes are retained in `results/learning_runs.csv` and `results/learning_summary.csv`.

Table 7 reports the complete factorial grid, so favorable settings are not selected for the main result. The retained-program proportions, cell sizes, and subsequent-eight-round net value and harm are in `results/learning_conditional.csv`. These conditional means remain descriptive because the retained program is selected using noisy evidence.

Table 6: Complete core net-value means. B: Biased; Bal: Balanced; N: Neyman; Mix: uniform fixed-program mixture; SR: successive rejection; O: discover once; R: repeated discovery. Each cell uses 128 trajectories. Full intervals and additional outcomes are supplied in the CSV files.

Environment	Memory	B	Bal	N	SR	O	R	Mix
Stationary harm	Reset	0.3967	0.4522	0.4539	0.4540	0.4366	0.4352	0.4343
Stationary harm	Cumulative	0.4692	0.4801	0.4805	0.4806	0.4793	0.4800	0.4766
Stationary harm	Window 8	0.4594	0.4792	0.4796	0.4800	0.4754	0.4779	0.4727
Workflow reversal	Reset	0.5260	0.5828	0.5830	0.5844	0.5644	0.5628	0.5639
Workflow reversal	Cumulative	0.4721	0.4803	0.4805	0.4829	0.4796	0.4835	0.4776
Workflow reversal	Window 8	0.5609	0.5758	0.5760	0.5786	0.5723	0.5821	0.5709
Uniform gain	Reset	0.6617	0.7141	0.7132	0.7162	0.6962	0.6941	0.6963
Uniform gain	Cumulative	0.7184	0.7215	0.7216	0.7218	0.7217	0.7219	0.7205
Uniform gain	Window 8	0.7166	0.7215	0.7216	0.7218	0.7210	0.7219	0.7199

Table 7: Repeated discovery minus discover-once at fixed total budget. All program shares and trial counts are shown. Entries are paired differences $\pm$ 95% Monte Carlo half-width; 96 fresh replicates per condition.

Program share	Trials/program	Stationary, cumulative	Reversal, Window 8
0.2	1	+0.0012 $\pm$ 0.0012	+0.0071 $\pm$ 0.0024
0.2	3	+0.0019 $\pm$ 0.0020	+0.0053 $\pm$ 0.0023
0.2	6	+0.0006 $\pm$ 0.0008	+0.0062 $\pm$ 0.0020
0.4	1	+0.0002 $\pm$ 0.0004	+0.0105 $\pm$ 0.0025
0.4	3	+0.0002 $\pm$ 0.0001	+0.0098 $\pm$ 0.0026
0.4	6	+0.0002 $\pm$ 0.0001	+0.0080 $\pm$ 0.0019
0.6	1	+0.0007 $\pm$ 0.0011	+0.0175 $\pm$ 0.0021
0.6	3	+0.0001 $\pm$ 0.0001	+0.0158 $\pm$ 0.0022
0.6	6	+0.0002 $\pm$ 0.0002	+0.0134 $\pm$ 0.0015

Table 8: Repeated discovery: retained-program percentages and subsequent eight-round outcomes conditional on retaining Balanced. Stationary and uniform gain use cumulative memory; reversal uses Window 8. B/Bal/N are Biased/Balanced/Neyman; n_{Bal} is the number retaining Balanced among 128 replicates. Harm is the percentage of rounds deploying a template worse than standard. Other memory conditions and discover-once results are supplied in the complete CSV.

Environment	Review	B (%)	Bal (%)	N (%)	n_{Bal}	Net given Bal	Harm (%)
Stationary	1	27.3	37.5	35.2	48	0.4747	0.00
Stationary	9	26.6	35.9	37.5	46	0.4819	0.00
Stationary	17	33.6	34.4	32.0	44	0.4819	0.00
Stationary	25	33.6	35.9	30.5	46	0.4819	0.00
Stationary	33	35.9	32.0	32.0	41	0.4819	0.00
Stationary	41	38.3	31.2	30.5	40	0.4819	0.00
Reversal	1	27.3	37.5	35.2	48	0.4747	0.00
Reversal	9	26.6	35.9	37.5	46	0.4783	0.00
Reversal	17	43.8	27.3	28.9	35	0.4819	0.00
Reversal	25	39.8	30.5	29.7	39	0.6188	0.00
Reversal	33	43.8	28.1	28.1	36	0.7219	0.00
Reversal	41	47.7	20.3	32.0	26	0.7219	0.00
Uniform gain	1	35.9	27.3	36.7	35	0.7219	0.00
Uniform gain	9	26.6	30.5	43.0	39	0.7219	0.00
Uniform gain	17	25.8	29.7	44.5	38	0.7219	0.00
Uniform gain	25	28.1	30.5	41.4	39	0.7219	0.00
Uniform gain	33	28.1	28.1	43.8	36	0.7219	0.00
Uniform gain	41	30.5	25.8	43.8	33	0.7219	0.00

D Supplementary audit-control model and stronger baseline

The supplementary audit model isolates longitudinal evidence acquisition under a fixed controller. It contains 160 periods of 32 tasks. Route A has base value 1 and error probability .04 (stationary) or alternating .04/.25 blocks of 40 periods; route B has base value .82 and error probability .08. An error costs 3, an audit costs .20, and an audit-rate change costs .50 per batch. Only audits reveal A’s errors, available next period; executing B can still be paired with acquiring an A audit. Observations update a sliding-window Beta estimate. The base prior is Beta(1, 9) and window 12.

The payoff-optimal routing threshold is .14, since $1 - 3p = .82 - 3(.08) = .58$. The heuristic adaptive controller audits at rate .40 when posterior $P(p > .14)$ lies between .10 and .90, and at .025 otherwise. Periodic dense auditing uses .40 for the first three periods of each 12-period cycle and .025 otherwise. The original experiment uses 400 seeds from 20260928, common across policies. Stationary net values are .86089 for fixed .025, .86395 for fixed .05, .85841 for fixed .10, and .87976 for no auditing. No auditing retains A under the base prior, the optimal route in that stationary case; audits cannot repair current outputs.

With slow shifts, adaptive auditing exceeds fixed .10 by .00857 per task (paired interval .00676–.01038). A separate 200-seed grid, beginning at 20360928 (an intentional offset of 100000 from the original seed range), crosses windows 6, 12, 24 with stationary, 40-period, and 10-period regimes. It includes no audit, fixed .05/.10/.40, periodic dense, and adaptive policies. The complete 54-row summary is supplied in `results/robustness_summary.csv`.

D.1 Finite-horizon expected value of sample information

The added baseline evaluates acquiring $m \in \{0, 1, 4, 16\}$ labels. Given current Beta parameters (a, b) , a beta-binomial predictive distribution over k observed errors gives posterior mean $(a + k)/(a + b + m)$. Let $L_\theta(p)$ be $1 - 3p$ if $p \leq \theta$ and .58 otherwise. The controller maximizes

$$h\{\mathbb{E}_k[L_\theta((a + k)/(a + b + m))] - L_\theta(a/(a + b))\} - \frac{[(.20 + g_c I_B)m + .50 I_{\text{change}}]}{32}. \quad (10)$$

I_{change} equals 1 when the audit rate changes and 0 otherwise. Both costs in the bracket are subtracted. I_B indicates executing B; g_c is the cost of generating a counterfactual A output to audit. The predictive sum is exact under the current Beta model. The controller approximates future value with horizon h and a sliding window, so it is a finite-horizon EVSI heuristic under drift.

Development fixes the prior at Beta(1, 9), the threshold at .14, and $g_c = 0$. Seeds 710000–710063 choose h from 1, 4, 8, 16 using stationary and slow-shift mean value; $h = 16$ is selected. Testing uses 96 independent seeds 810000–810095, three priors, thresholds .14/.16, generation costs $g_c = 0/.20$, two environments, and three policies: 6912 trajectories. Threshold .16 is a misspecified routing-rule sensitivity under unchanged payoffs. Nonzero g_c charges selectively generated A outputs while B is executed; it is distinct from a common cost charged for producing every candidate under every policy.

Table 9 reports all prior/threshold means at $g_c = 0$. The companion CSV contains both costs, marginal intervals, and paired contrasts. EVSI’s performance depends strongly on the prior: with no initial feedback, information acquisition itself can stall. Four regression cases match the preserved scalar implementation to 10^{-12} .

Table 9: Supplementary audit sensitivity at zero additional candidate-generation cost. Columns give net-value means over 96 trajectories; F: fixed .10, A: adaptive heuristic, E: EVSI. Full uncertainty and $g_c = .20$ results are supplied in the CSV.

Prior ($a : b$)	Threshold	Stationary			Slow shifts		
		F	A	E	F	A	E
1:1	0.14	0.8479	0.8530	0.8338	0.6616	0.6753	0.6758
1:1	0.16	0.8523	0.8571	0.8342	0.6597	0.6729	0.6757
1:9	0.14	0.8581	0.8649	0.8656	0.6620	0.6706	0.6599
1:9	0.16	0.8592	0.8686	0.8660	0.6515	0.6642	0.6526
1:19	0.14	0.8595	0.8720	0.8795	0.6468	0.6571	0.5673
1:19	0.16	0.8595	0.8735	0.8795	0.6263	0.6396	0.5673